PROYECTO ECOSORT · CLASIFICACIÓN DE RESIDUOS

Reporte de pruebas de modelos ligeros

Comparación de MobileNetV3-Large, EfficientNetB0 y ShuffleNetV2 (ImageNet) en la Raspberry Pi 4 Model B, con la Raspberry Pi 5 como referencia

DISPOSITIVO DE DESPLIEGUE
Raspberry Pi 4 Model B Rev 1.5

CONJUNTO DE PRUEBA
4,534 imágenes · 5 clases

FECHA DE LAS PRUEBAS
14 de octubre de 2026

MobileNetV3-Large

Recomendado

Accuracy	**96.27%**
Latencia media	**52.28 ms**
Cuadros/s	**19.1**
Archivo .tflite	**3.24 MB**

EfficientNetB0

Accuracy	**96.52%**
Latencia media	**131.12 ms**
Cuadros/s	**7.6**
Archivo .tflite	**4.50 MB**

ShuffleNetV2 (ImageNet)

Accuracy	**94.27%**
Latencia media	**38.15 ms**
Cuadros/s	**26.2**
Archivo .tflite	**1.50 MB**

Cifras medidas en la Raspberry Pi 4 con modelos .tflite cuantizados, 1 hilo de inferencia (runtime ai_edge_litert).

Contenido

Introducción 2

1. Resumen 3

2. Entorno de pruebas 4

3. Resultados en la Raspberry Pi 4 5

4. Raspberry Pi 4 frente a Raspberry Pi 5 16

5. Cumplimiento de H1 en la Pi 4 18

6. Prueba complementaria: 4 hilos de inferencia 19

7. Conclusiones 21

8. Evidencias generadas 22

9. Cómo se hicieron y cómo reproducir las pruebas 25

Introducción

Repetición, en una **Raspberry Pi 4 Model B**, de las mismas pruebas que se hicieron en la Raspberry Pi 5 (`docs/reporte_benchmark_modelos.md`): los tres clasificadores ligeros de residuos (5 clases: `aluminio`, `carton`, `otros`, `plastico`, `vidrio`) en formato `.tflite`, sobre el mismo conjunto de prueba, con el mismo script, los mismos parámetros y el mismo runtime.

El objetivo es saber cómo se comportan los modelos en un hardware más modesto y si las conclusiones obtenidas en la Pi 5 siguen siendo válidas.

Todas las gráficas y capturas de este reporte (carpeta `reports/figures/raspberry4/`) provienen de la Raspberry Pi 4. Las cifras de la Pi 5 aparecen solo como referencia en las tablas comparativas.

Fecha de las pruebas: 14 de octubre de 2026.

1. Resumen

Modelo	Accuracy en Pi 4	Errores	F1 macro	Latencia media en Pi 4	Cuadros/s	Latencia en Pi 5	Pi 4 / Pi 5
MobileNetV3-Large	96.27%	169 / 4,534	0.9604	**52.28 ms**	19.1	15.50 ms	×3.4
EfficientNetB0	**96.52%**	158 / 4,534	**0.9631**	131.12 ms	7.6	45.40 ms	×2.9
ShuffleNetV2 (ImageNet)	94.27%	260 / 4,534	0.9392	**38.15 ms**	26.2	9.35 ms	×4.1

- **Velocidad:** la Pi 4 es entre 2.9 y 4.1 veces más lenta que la Pi 5. El orden entre modelos no cambia: ShuffleNetV2 es el más rápido, luego MobileNetV3-Large y al final EfficientNetB0.
- **Hipótesis H1 (latencia < 200 ms):** los tres modelos la cumplen, pero **EfficientNetB0 queda justo**. Su peor inferencia tardó 169.4 ms (margen ×1.2) y, si se suma el preprocesamiento medio de la imagen (19.3 ms en la Pi 4), llega a unos 189 ms, a 11 ms del límite. MobileNetV3-Large tiene un peor caso de 84.7 ms (margen ×2.4) y ShuffleNetV2 de 53.8 ms (×3.7).
- **Precisión:** prácticamente igual que en la Pi 5 (diferencias de −0.20 a +0.09 puntos). MobileNetV3-Large y EfficientNetB0 siguen siendo estadísticamente equivalentes (McNemar $p = 0.42$). ShuffleNetV2 sigue siendo significativamente peor ($p < 10^{-9}$).
- **Estabilidad:** en 5 minutos de inferencia continua por modelo, la deriva de latencia no pasó de +0.82%, la RAM del proceso no creció más de 0.03 MB y no hubo throttling ni bajo voltaje (`0x0`). La temperatura máxima fue de 50.6 °C.
- **Cuadros por segundo:** con un solo hilo ningún modelo llega a los 30 cuadros/s de una cámara típica (MobileNetV3: 19.1). Con 4 hilos, MobileNetV3 llega a 31.0 cuadros/s, pero ocupa los cuatro núcleos.

Modelo recomendado para una Pi 4: MobileNetV3-Large, igual que en la Pi 5. Tiene la misma precisión que EfficientNetB0, es 2.5 veces más rápido y cumple H1 con margen aun sumando el preprocesamiento. En la Pi 4, **EfficientNetB0 deja de ser una alternativa razonable**: 7.6 cuadros/s y un peor caso cerca del límite de H1. La Pi 4 sirve como hardware mínimo para el sistema, pero con menos holgura que la Pi 5.

2. Entorno de pruebas

	Raspberry Pi 4 (esta prueba)	Raspberry Pi 5 (referencia)
Placa	Raspberry Pi 4 Model B Rev 1.5	Raspberry Pi 5 Model B Rev 1.1
Procesador	Broadcom BCM2711, 4 × Cortex-A72 a 1.8 GHz (ARMv8.0-A)	Broadcom BCM2712, 4 × Cortex-A76 a 2.4 GHz (ARMv8.2-A)
RAM	8 GB	16 GB
Sistema	Debian 13 (trixie), kernel 6.18.50+rpt-rpi-v8	Debian 13 (trixie), kernel 6.18.50+rpt-rpi-2712
Python	3.13.5	3.13.5
Runtime	`ai-edge-litert` 2.2.0 (`.tflite` en CPU)	`ai-edge-litert` 2.2.0 (`.tflite` en CPU)
Hilos de inferencia	valor por defecto del runtime (1 hilo*); también se probó con 4 (sección 6)	igual
Gobernador de CPU	`ondemand`	`ondemand`
Refrigeración	sin ventilador registrado en el sistema (solo el sensor `cpu_thermal`)	ventilador activo

* El JSON registra `num_threads: null` (valor por defecto del runtime). Que sea 1 hilo se deduce de que la CPU total se mantuvo en 25%, es decir, un núcleo de cuatro al 100%, y de que con `--num-threads 4` la latencia sí cambia.

Instalación en la Pi 4: todo se copió a una sola carpeta, `~/ecosort_benchmark/`, con el script, los 3 modelos, el set de prueba (922 MB) y un entorno virtual propio (`venv/`). En ese entorno solo se instaló `ai-edge-litert` 2.2.0; `numpy` 2.2.4, `pillow` 11.1.0, `psutil` 7.0.0 y `rich` 13.9.4 ya venían en el sistema. No se instaló nada fuera de esa carpeta.

Conjunto de prueba: el mismo que en la Pi 5, `data/raw/waste_classification/test`, con **4,534 imágenes**: aluminio 888, cartón 730, otros 1,164, plástico 1,268 y vidrio 484. Se verificó que las tres corridas (Pi 4, Pi 5 y laptop) evaluaron las mismas imágenes en el mismo orden.

Preprocesamiento: el mismo que en el entrenamiento: RGB, redimensionado a 224×224 con interpolación *nearest* y pixeles `float32` en rango 0-255 (la normalización va dentro de cada red).

Modelos: los mismos archivos que en la Pi 5: `modelo_residuos_rpi_V3_05.tflite` (3.24 MB), `modelo_residuos_EfficientNetB0_v1.tflite` (4.50 MB) y `modelo_residuos_shufflenetV2_imagenet.tflite` (1.50 MB).

3. Resultados en la Raspberry Pi 4

Corrida oficial: `tests/benchmark_visual.py` con los parámetros por defecto, igual que en la Pi 5 (todas las imágenes, 10 inferencias de calentamiento, 5 minutos de estabilidad por modelo y 60 s de enfriamiento antes de cada modelo). Duró 37 min 28 s, del 14/10/2026 00:03 a las 00:40. Datos crudos: `reports/benchmark_rpi4/resultados_rpi4_1hilo.json`; registro de la terminal: `reports/benchmark_rpi4/log_rpi4_1hilo.txt`.

3.1 Precisión de clasificación

Modelo	Accuracy	IC 95% (Wilson)	Errores	Precisión macro	Recall macro	F1 macro
MobileNetV3-Large	96.27%	95.68 – 96.79%	169	0.9625	0.9587	0.9604
EfficientNetB0	96.52%	95.94 – 97.01%	158	0.9634	0.9631	0.9631
ShuffleNetV2 (ImageNet)	94.27%	93.55 – 94.91%	260	0.9422	0.9370	0.9392

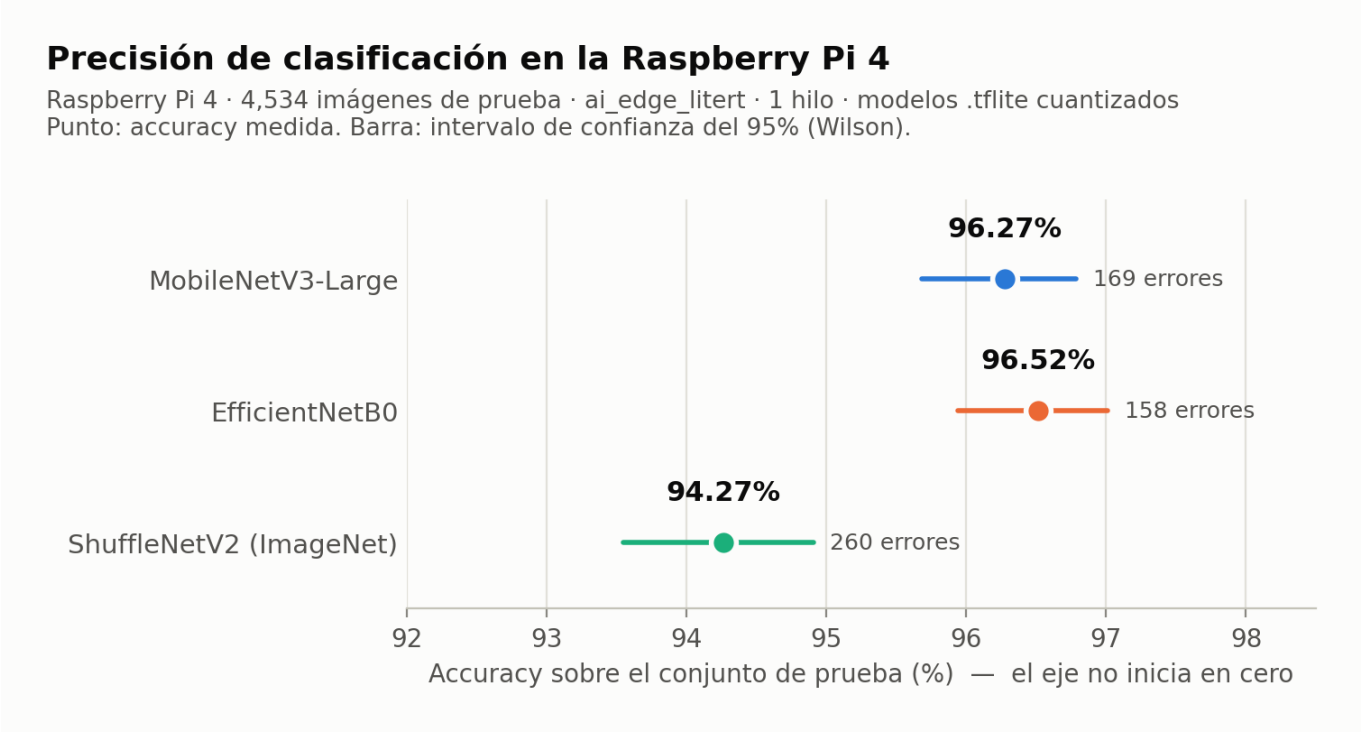


Figura 1. Accuracy de los modelos

Significancia estadística (prueba de McNemar exacta, mismas 4,534 imágenes):

Comparación	Solo acierta el primero	Solo acierta el segundo	p	¿Diferencia significativa?
MobileNetV3-Large vs EfficientNetB0	72	83	0.42	No
MobileNetV3-Large vs ShuffleNetV2	157	66	9.9×10^{-10}	Sí
EfficientNetB0 vs ShuffleNetV2	161	59	4.0×10^{-12}	Sí

Accuracy por clase:

Clase (n)	MobileNetV3-Large	EfficientNetB0	ShuffleNetV2 (ImageNet)
aluminio (888)	**97.64%**	97.52%	95.50%
carton (730)	97.26%	**98.22%**	97.40%
otros (1,164)	**97.68%**	96.48%	95.53%
plastico (1,268)	95.27%	**96.14%**	93.30%
vidrio (484)	91.53%	**93.18%**	86.78%

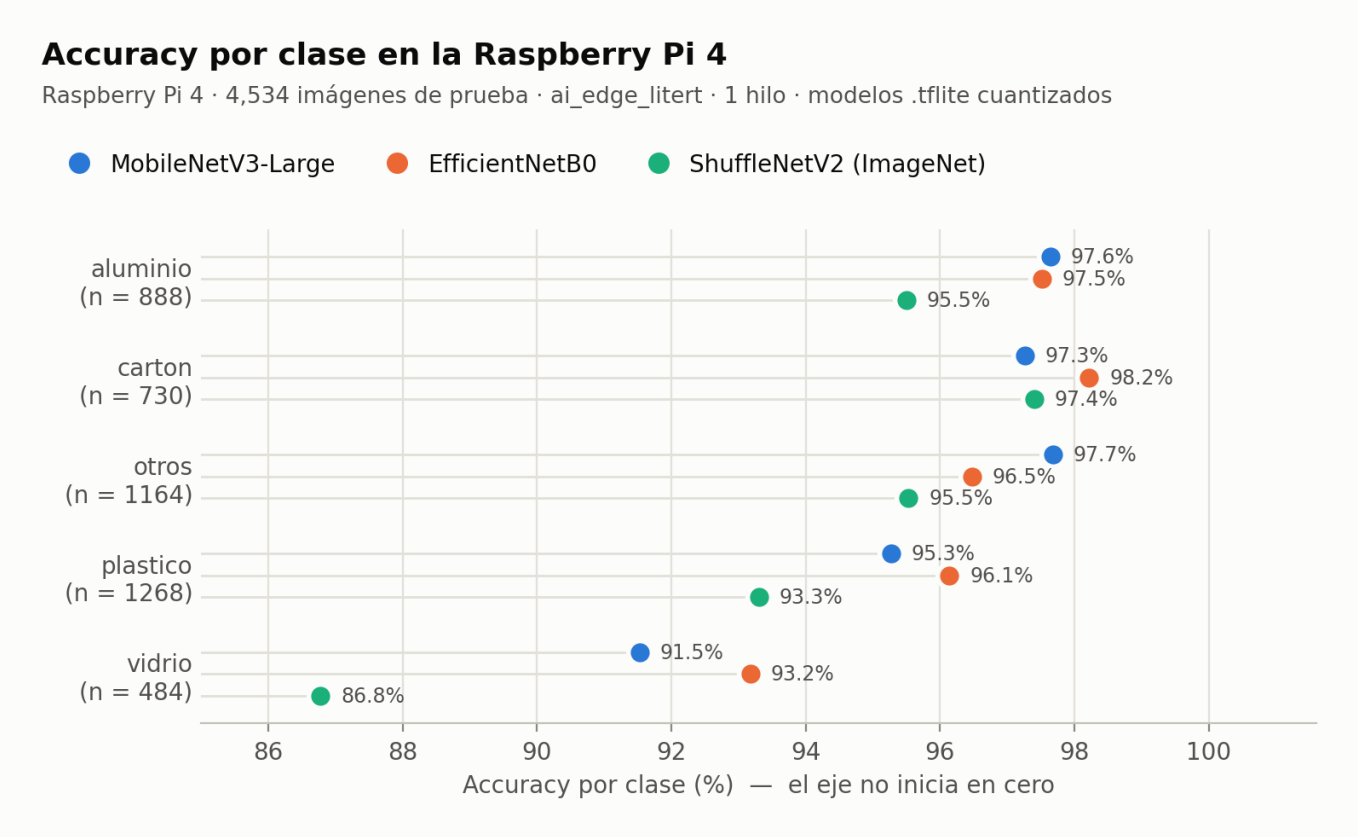


Figura 2. Accuracy por clase

Matrices de confusión (filas = clase real, columnas = clase predicha; orden aluminio, carton, otros, plastico, vidrio):

```
MobileNetV3-Large            EfficientNetB0               ShuffleNetV2 (ImageNet)
[ 867    1   10    7    3]   [ 866    7    7    6    2]   [ 848    7   20    7    6]
[   7  710    7    6    0]   [   5  717    6    1    1]   [   9  711    5    5    0]
[  11   10 1137    6    0]   [  12   12 1123   13    4]   [  18   11 1112   17    6]
[  22    8   15 1208   15]   [  16    8   10 1219   15]   [  33    9   26 1183   17]
[   9    1    9   22  443]   [   5    0    5   23  451]   [  11    2   15   36  420]
```

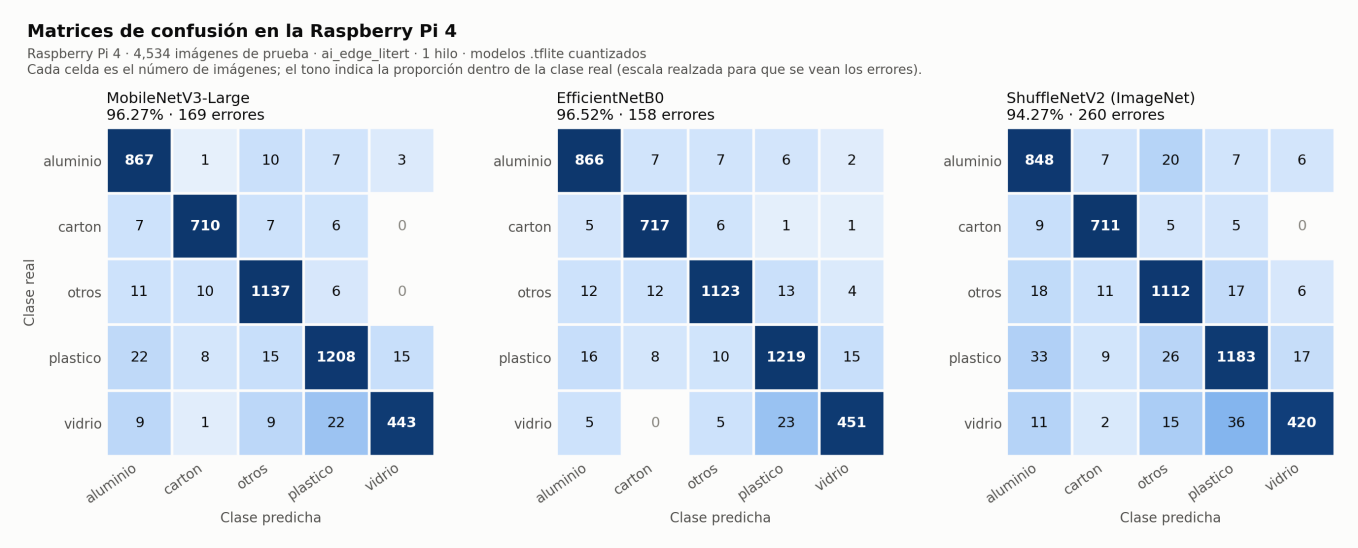


Figura 3. Matrices de confusión

Observaciones:

- Igual que en la Pi 5, `vidrio` es la clase más difícil y su error principal es confundirse con `plastico` (22, 23 y 36 casos).
- De las 4,534 imágenes, 4,167 las aciertan los tres modelos, 215 las falla solo uno, 84 las fallan dos y **68 las fallan los tres**. Las 10 imágenes de "casos difíciles" que se mostraron en el reporte de la Pi 5 también fallan en los tres modelos en la Pi 4.

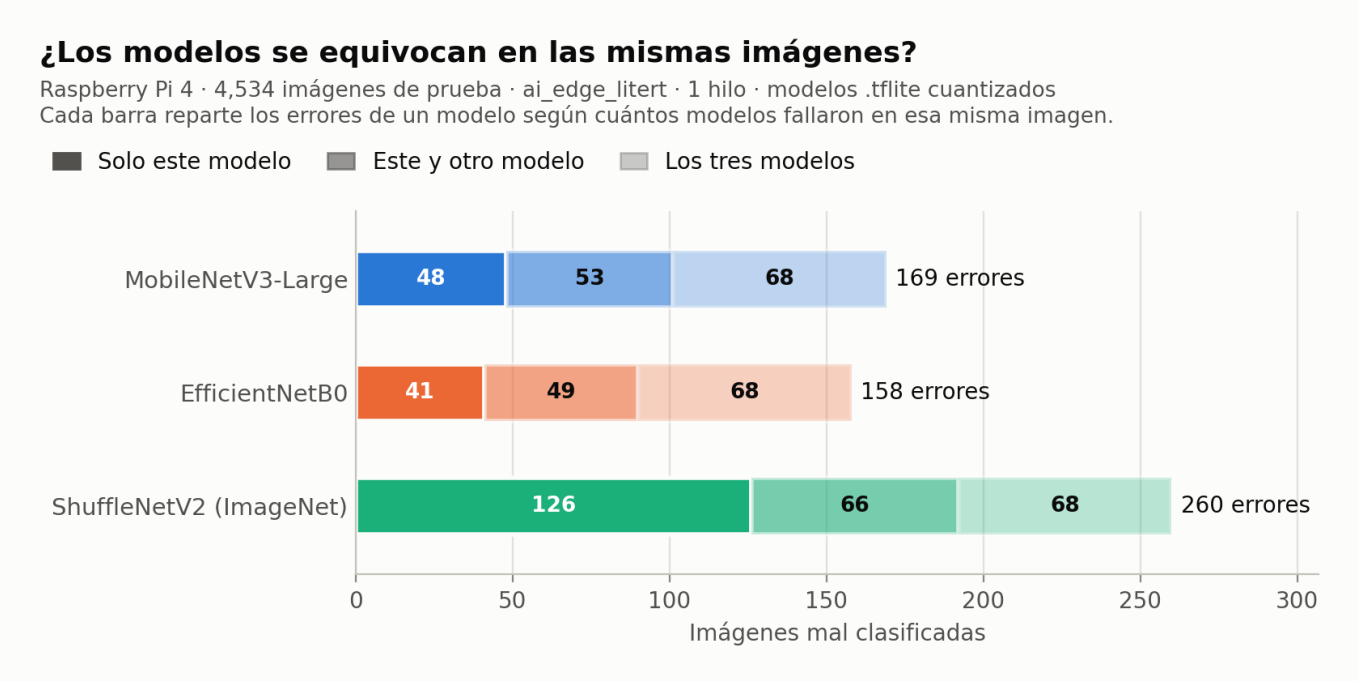


Figura 4. Coincidencia de errores

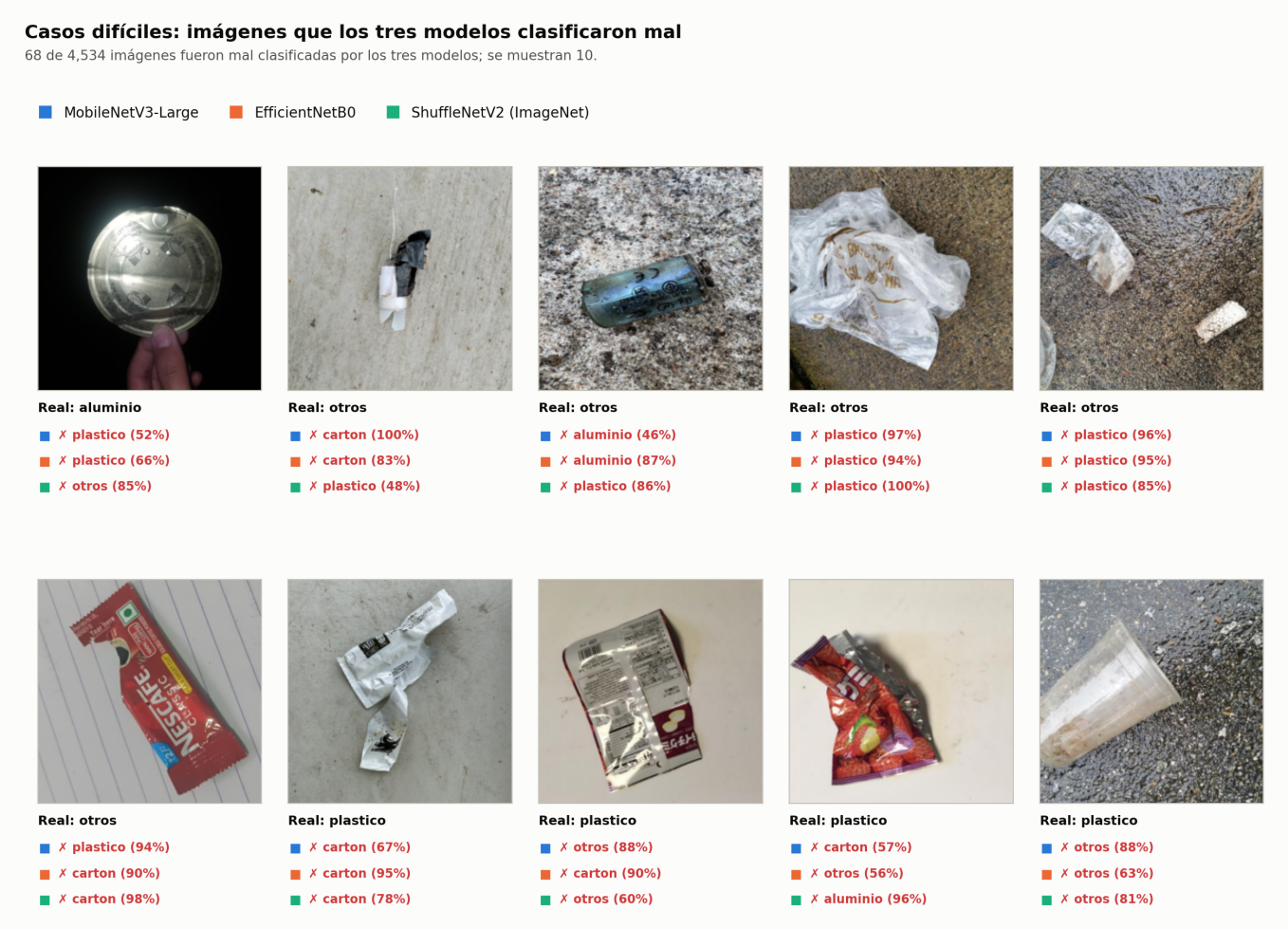


Figura 5. Casos difíciles

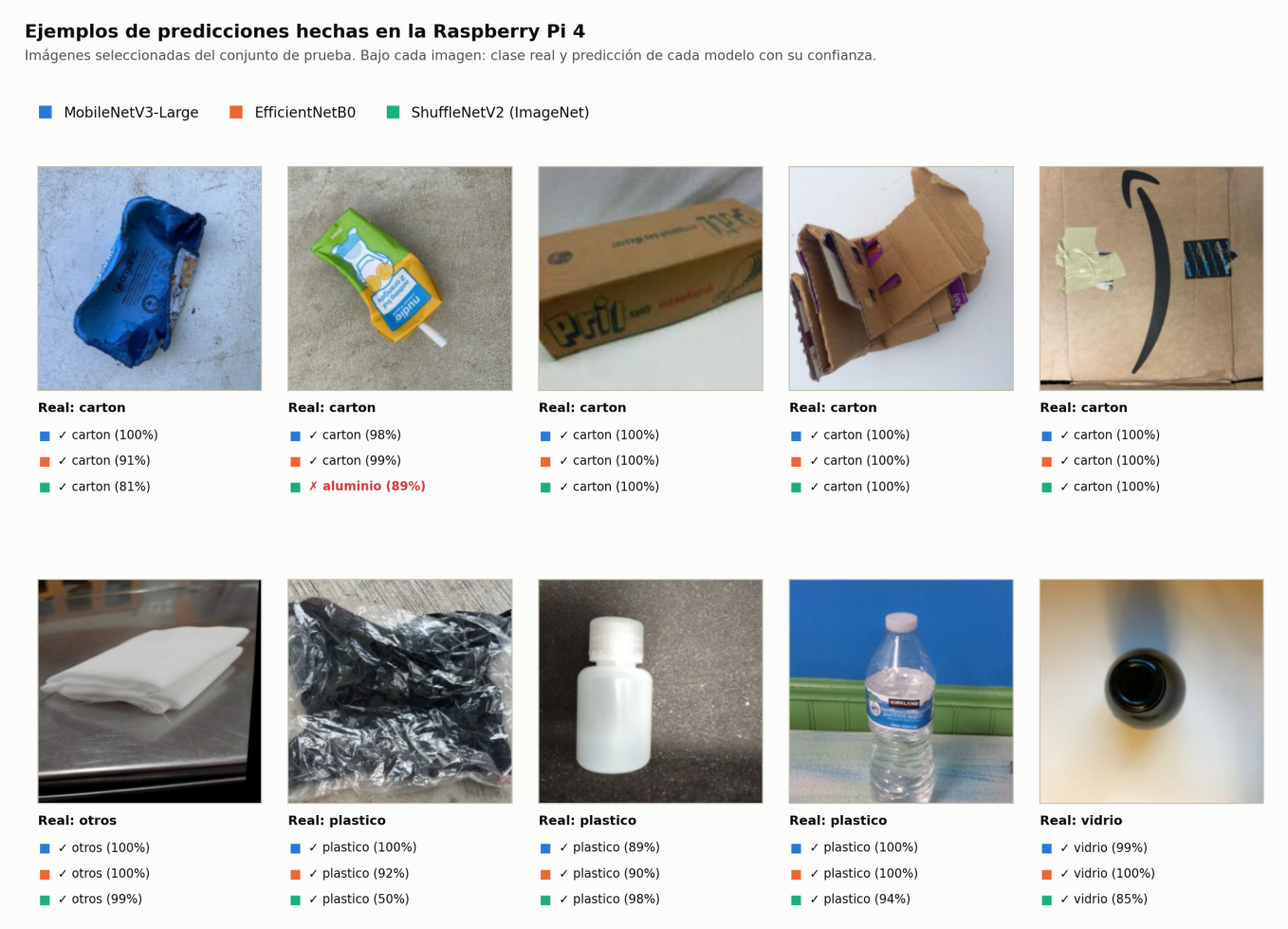


Figura 6. Ejemplos de predicciones

3.2 Latencia de inferencia

Se cronometra `set_tensor + invoke + get_tensor` de cada imagen y se descartan las primeras 10 inferencias como calentamiento (4,524 mediciones por modelo).

Modelo	Media	Desv. est.	Mín	P50	P95	P99	Máx	Cuadros/s	Margen frente a H1 (peor caso)
MobileNetV3-Large	52.28 ms	1.54	51.64	51.96	53.99	55.60	84.75	19.1	×2.4
EfficientNetB0	131.12 ms	2.61	129.34	130.58	134.62	137.52	169.43	7.6	×1.2
ShuffleNetV2 (ImageNet)	38.15 ms	0.86	37.63	37.95	39.69	40.22	53.76	26.2	×3.7

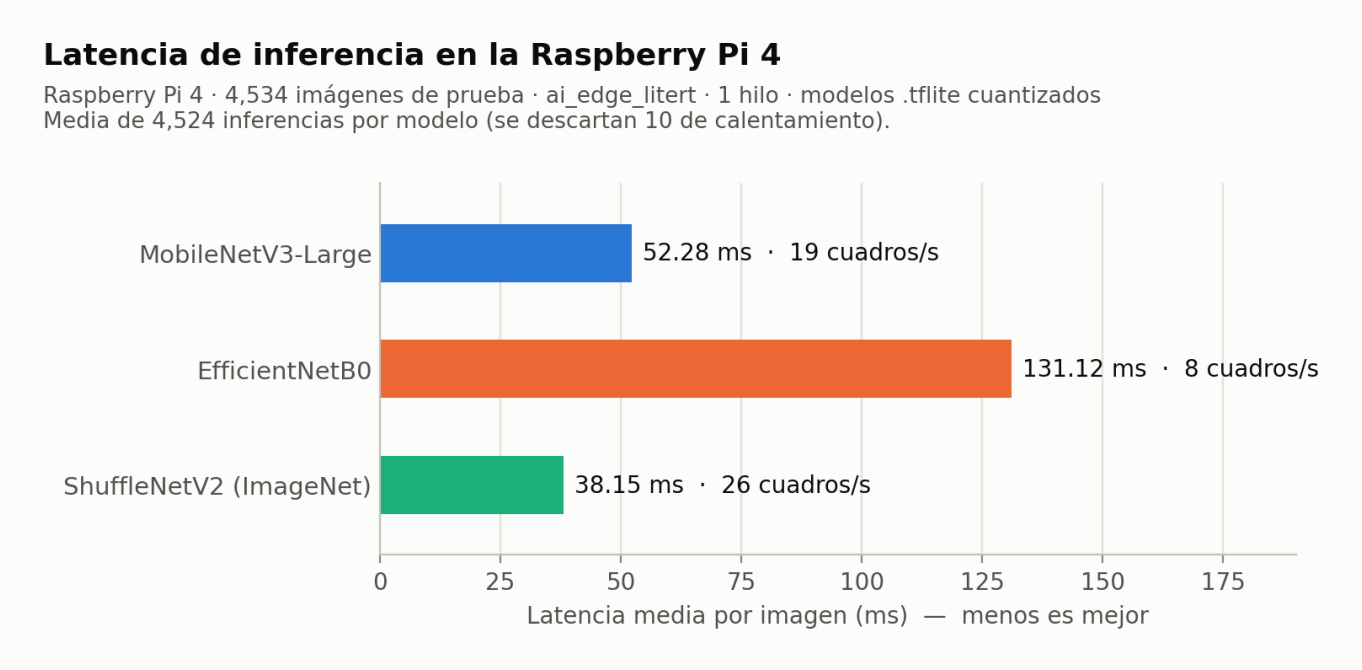


Figura 7. Latencia media

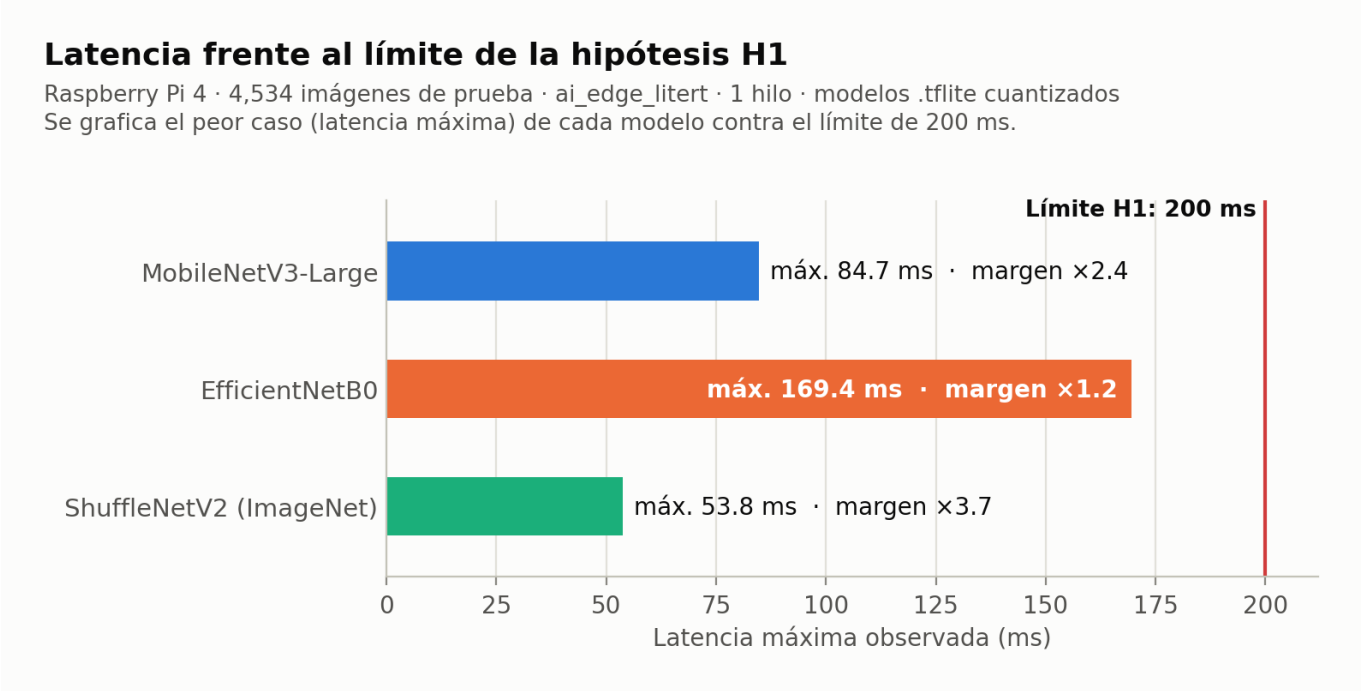


Figura 8. Latencia frente a H1

La latencia es estable, aunque algo menos que en la Pi 5 (desviación estándar de 0.9 a 2.6 ms, frente a 0.03–0.06 ms). Los picos son raros: solo 21, 16 y 10 inferencias (menos del 0.5%) superan en más de 20% a la mediana. Esos picos explican los máximos de 84.7, 169.4 y 53.8 ms.

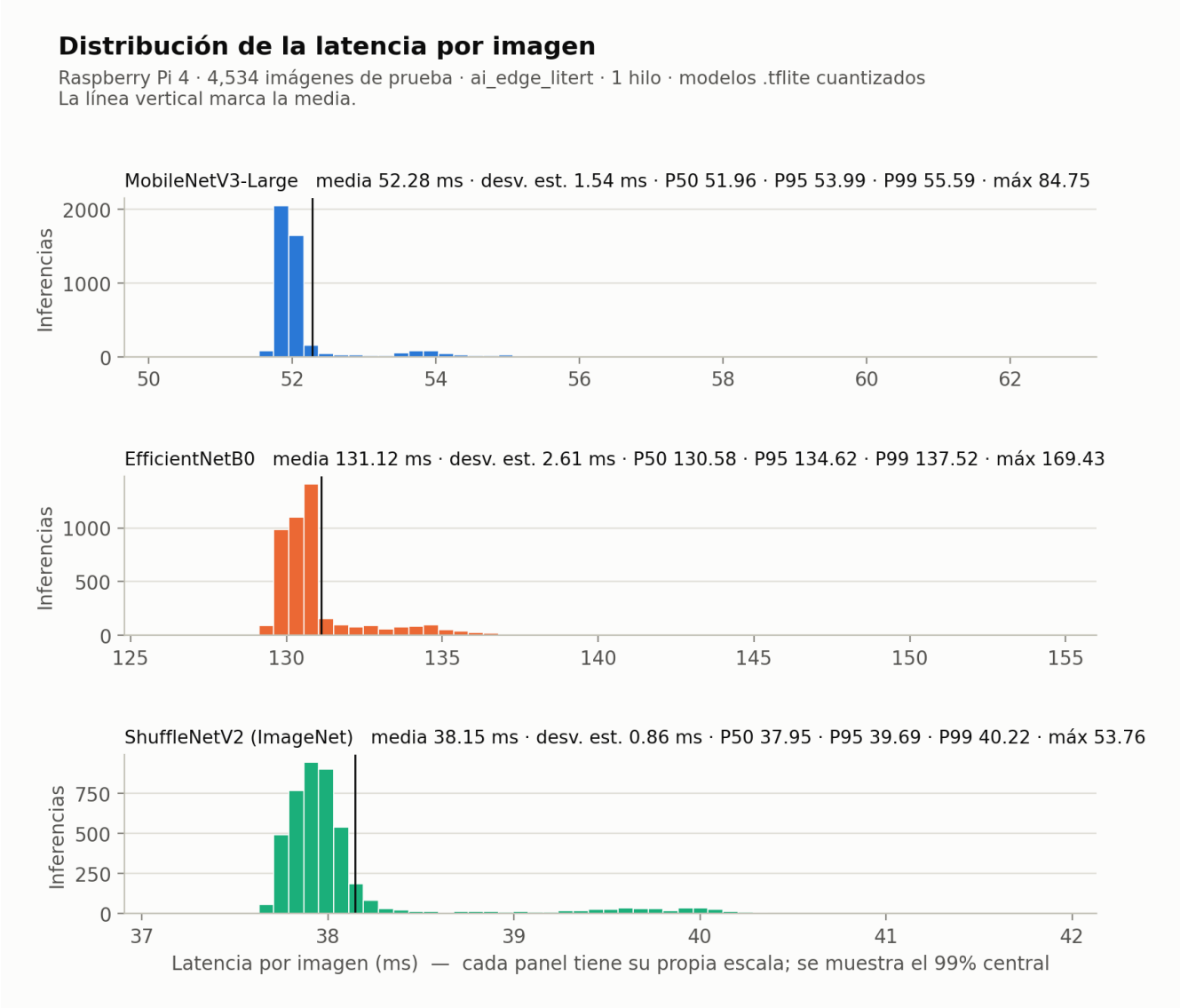


Figura 9. Distribución de la latencia

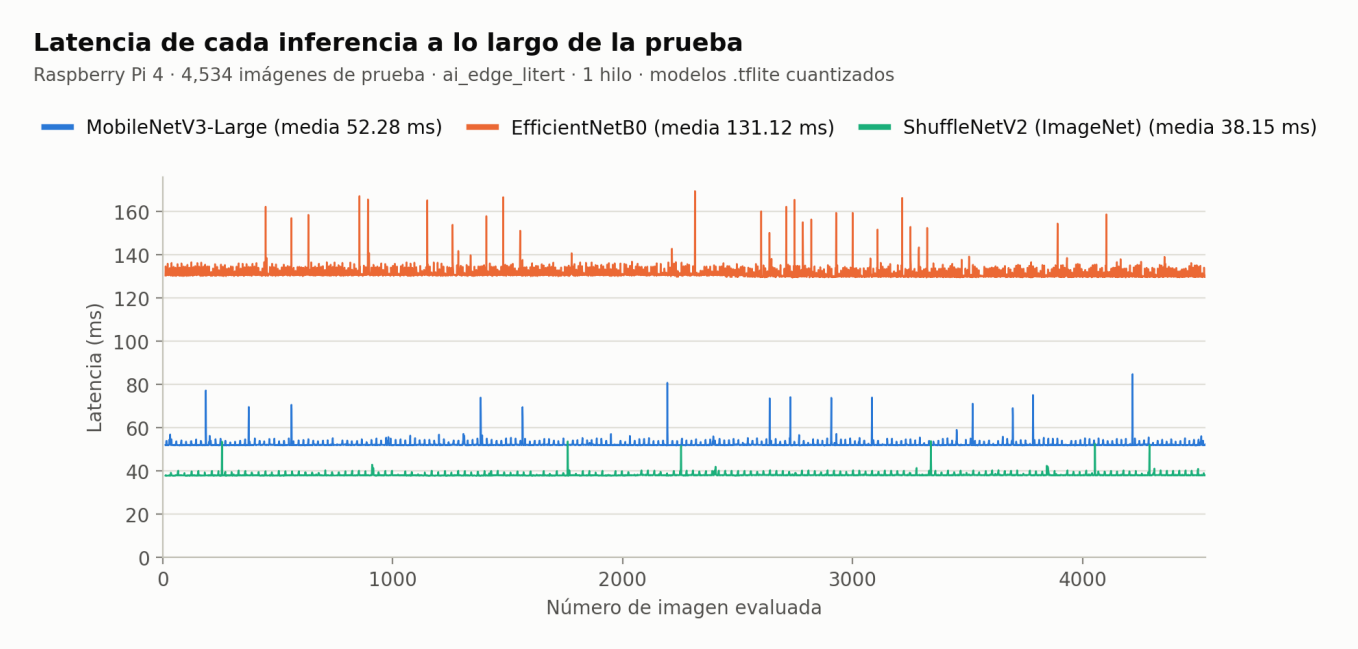


Figura 10. Latencia por imagen

Costo del preprocesamiento: decodificar y redimensionar las 4,534 imágenes tomó 87.4 s, unos **19.3 ms por imagen** (en la Pi 5 eran 8.2 ms). Este tiempo queda fuera de la latencia medida, pero en el sistema final se suma a la inferencia:

Modelo	Inferencia media + preprocesamiento	Peor inferencia + preprocesamiento medio	Frente a H1 (200 ms)
MobileNetV3-Large	≈ 72 ms	≈ 104 ms	holgado
EfficientNetB0	≈ 150 ms	≈ 189 ms	**muy justo**
ShuffleNetV2 (ImageNet)	≈ 57 ms	≈ 73 ms	holgado

Con la cámara, el preprocesamiento debería costar menos, porque los cuadros llegan ya con resolución reducida. Aun así, la captura del cuadro y el control de los actuadores también consumen tiempo, así que con EfficientNetB0 en la Pi 4 no queda margen real.

3.3 Estabilidad (5 minutos de inferencia continua por modelo)

Modelo	Inferencias	Inf/s	CPU media (máx)	Deriva de latencia	Δ RAM	Temp. media / máx	Frecuencia	Throttling
MobileNetV3-Large	5,680	18.9	25.2% (31.8%)	+0.03%	+0.03 MB	48.4 / 50.6 °C	1800 MHz*	ninguno (`0x0`)
EfficientNetB0	2,265	7.5	25.2% (32.1%)	+0.82%	+0.02 MB	39.7 / 41.9 °C	1800 MHz fija	ninguno (`0x0`)
ShuffleNetV2 (ImageNet)	7,730	25.8	25.2% (32.4%)	−0.16%	+0.01 MB	39.5 / 41.4 °C	1800 MHz*	ninguno (`0x0`)

* Se mantuvo en 1800 MHz salvo en 2 muestras (MobileNetV3) y 1 muestra (ShuffleNetV2) de 300, en las que el gobernador `ondemand` la bajó un momento a 1200 MHz.

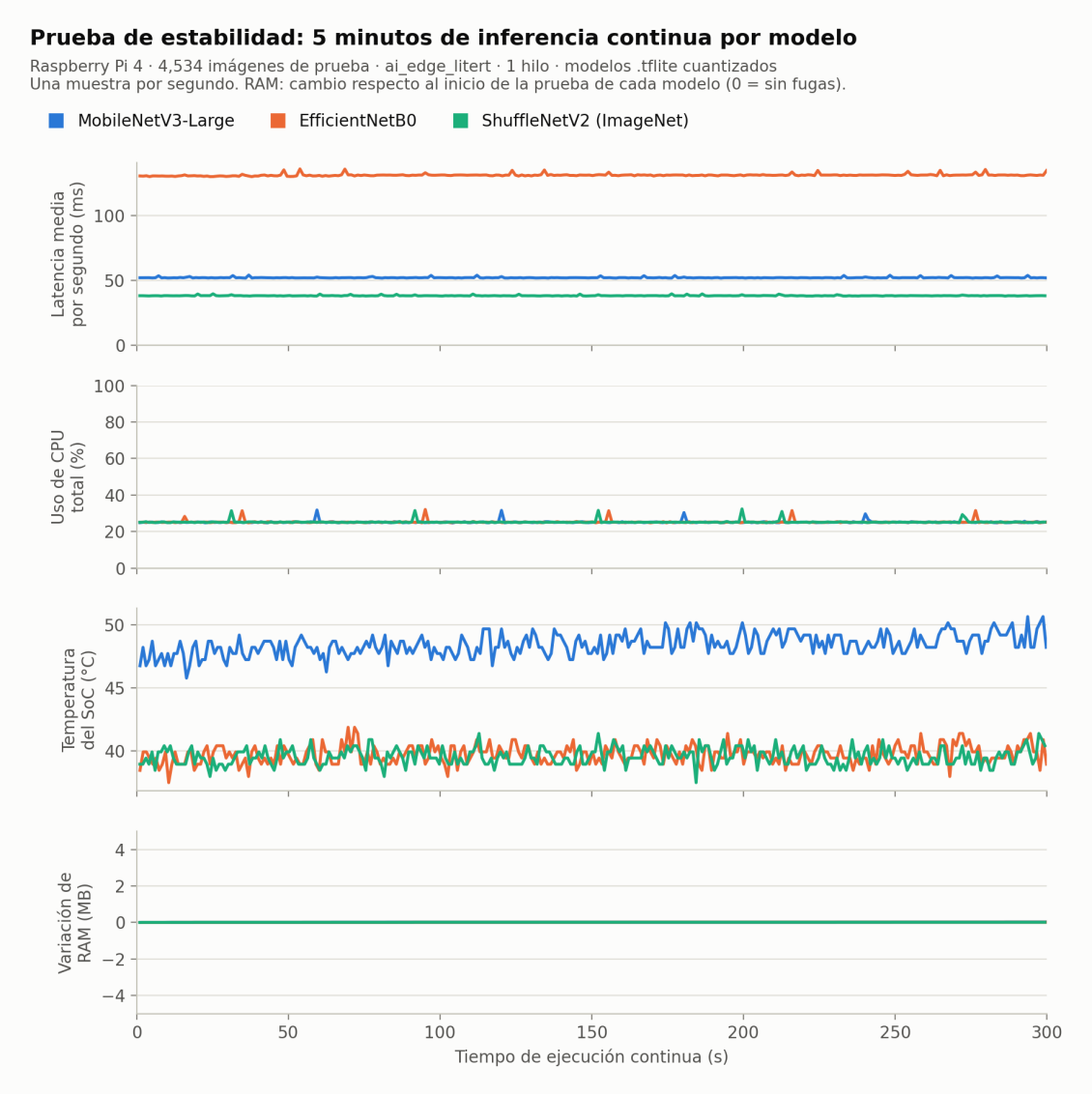


Figura 11. Estabilidad

- **CPU al 25%:** igual que en la Pi 5, con un hilo el intérprete ocupa un núcleo de cuatro y deja libres los otros tres. Los pequeños picos de ~32% se repiten cada ~60 s en los tres modelos, así que corresponden a alguna tarea periódica del sistema y no a la inferencia.
- **Temperatura:** muy por debajo de los 80 °C en que el firmware empieza a reducir la frecuencia. MobileNetV3 se probó más caliente (48 °C) que los otros dos (40 °C) aunque es el modelo más ligero de los dos primeros. La diferencia no depende del modelo y probablemente se deba a cambios de temperatura ambiente durante los 37 minutos de la prueba.
- **RAM:** la memoria del proceso cambió como máximo 0.03 MB en 5 minutos, así que no hay fugas. El proceso ocupa ~830 MB porque mantiene en memoria las 4,534 imágenes preprocesadas; lo que ocupa el modelo es mucho menos (sección 3.4).

3.4 Consumo de recursos

Modelo	`.tflite`	RAM del modelo cargado*	Tiempo de carga**	CPU en inferencia continua
MobileNetV3-Large	3.24 MB	16.9 MB	108 ms	25.2%
EfficientNetB0	4.50 MB	33.9 MB	125 ms	25.2%
ShuffleNetV2 (ImageNet)	1.50 MB	12.0 MB	51 ms	25.2%

* Medida en un proceso aislado: RAM del proceso con el modelo cargado y 30 inferencias ejecutadas, menos la del proceso vacío (40.8 MB). Es casi idéntica a la de la Pi 5 (17.1, 34.2 y 12.2 MB).

** Incluye leer el archivo del disco (USB) y crear el delegado XNNPACK. En la demostración posterior, con el archivo ya en caché, MobileNetV3 cargó en 45 ms.

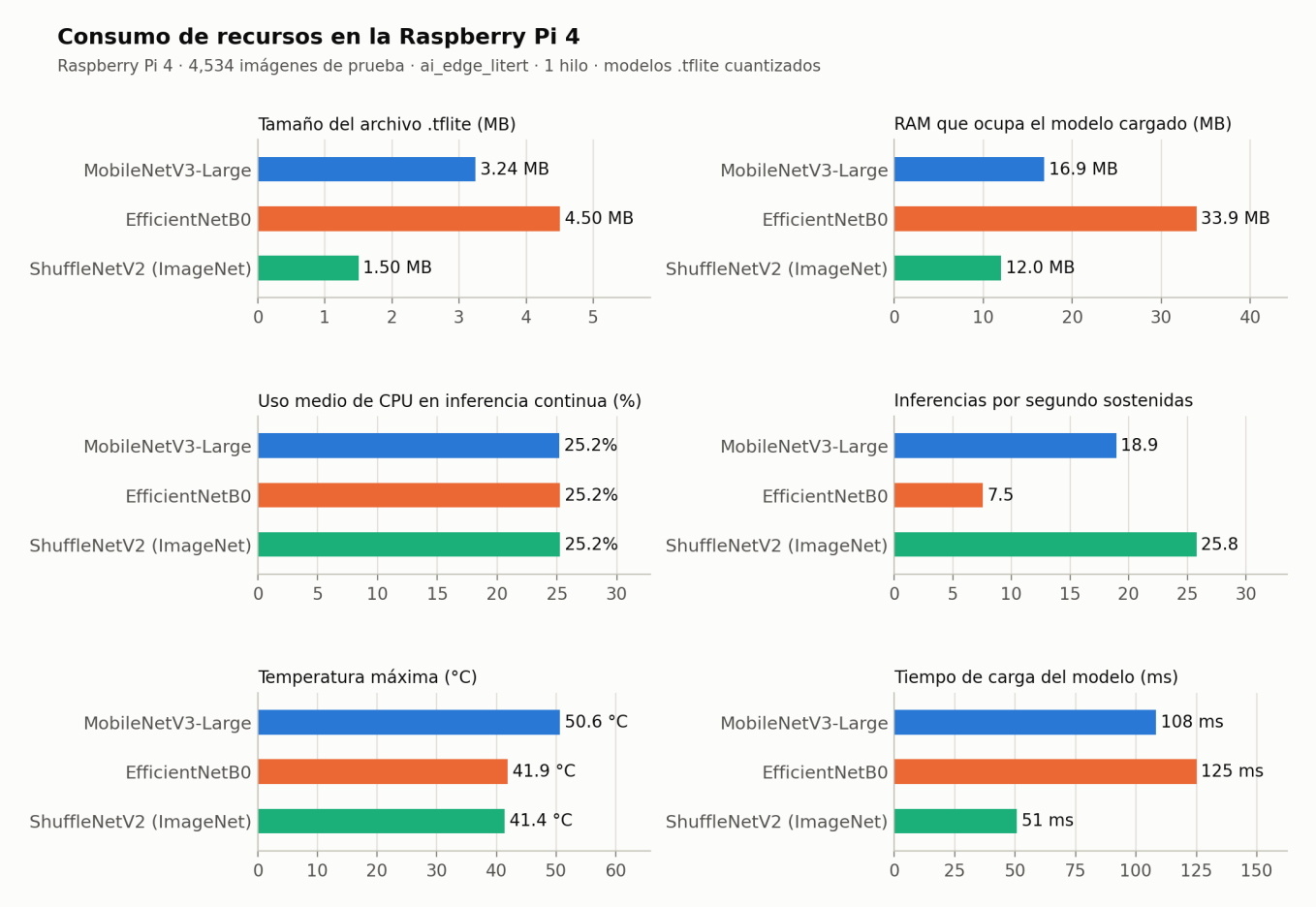


Figura 12. Recursos

3.5 Compromiso entre precisión y velocidad

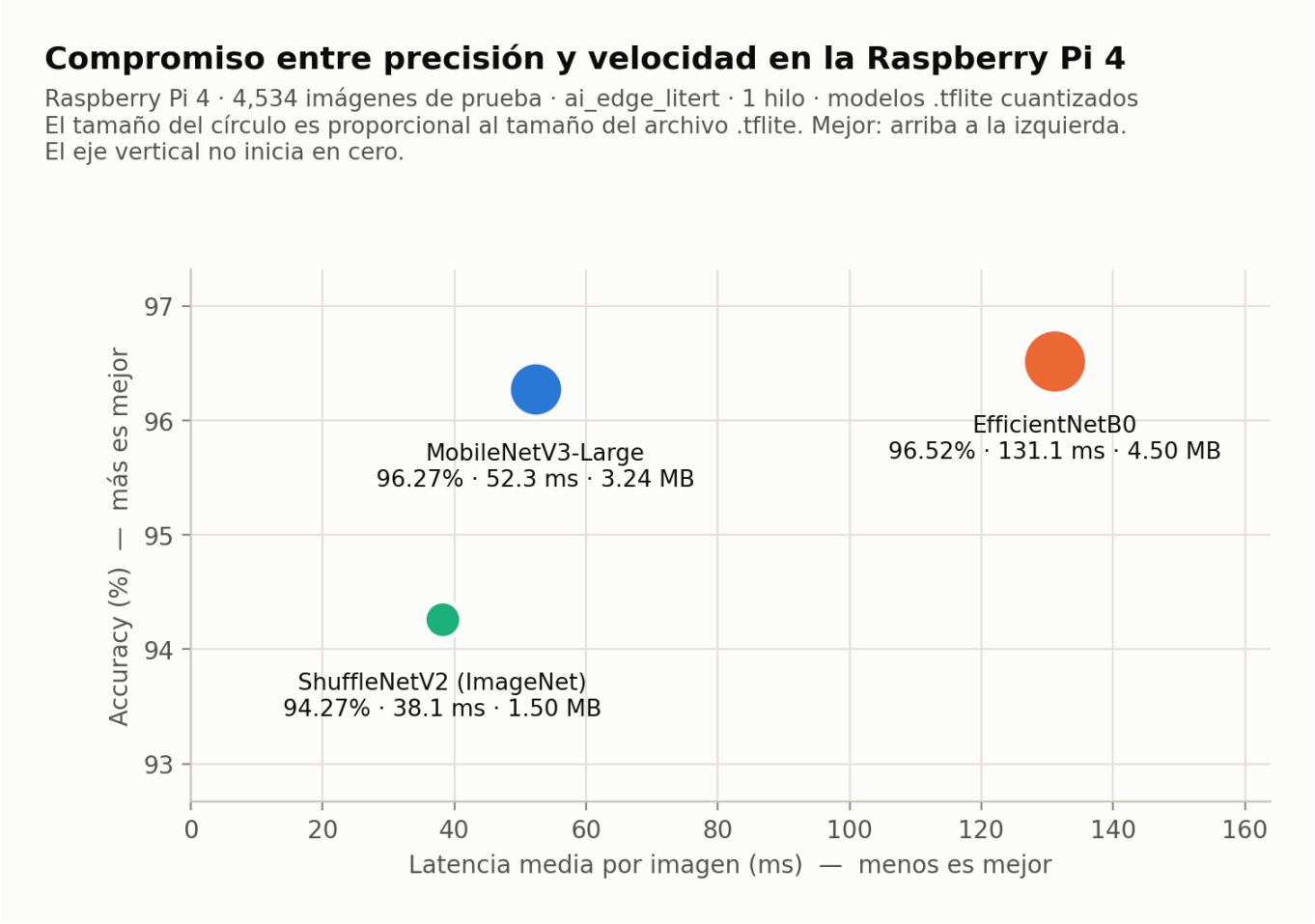


Figura 13. Precisión vs latencia

4. Raspberry Pi 4 frente a Raspberry Pi 5

4.1 Velocidad

Modelo	Pi 5 (1 hilo)	Pi 4 (1 hilo)	Pi 4 / Pi 5	Inferencias/s sostenidas Pi 5 → Pi 4
MobileNetV3-Large	15.50 ms	52.28 ms	×3.37	63.9 → 18.9
EfficientNetB0	45.40 ms	131.12 ms	×2.89	21.9 → 7.5
ShuffleNetV2 (ImageNet)	9.35 ms	38.15 ms	×4.08	105.2 → 25.8

- La diferencia de frecuencia (2.4 frente a 1.8 GHz) solo explica un factor de 1.33. El resto viene de la microarquitectura: el Cortex-A76 ejecuta más instrucciones por ciclo, tiene instrucciones de producto punto (`SDOT`) y aritmética FP16 que el Cortex-A72 no tiene, y una jerarquía de caché y memoria más rápida.
- **ShuffleNetV2 es el que más pierde (×4.1).** En la Pi 5 era 1.66 veces más rápido que MobileNetV3; en la Pi 4 solo 1.37 veces. La explicación más probable es que sus operaciones de división y mezcla de canales (*channel split/shuffle*) mueven datos sin hacer cálculo, y la Pi 4 tiene menos ancho de banda de memoria.
- **EfficientNetB0 es el que menos pierde (×2.9),** pero partía de muy atrás. En la Pi 4 sigue siendo 2.5 veces más lento que MobileNetV3.

4.2 Precisión: el mismo `.tflite` en cuatro entornos

Modelo	`.keras` laptop (GPU)	`.tflite` laptop (x86)	`.tflite` Pi 5 (Cortex-A76)	`.tflite` Pi 4 (Cortex-A72)
MobileNetV3-Large	96.52% (158 err.)	96.27% (169 err.)	96.47% (160 err.)	96.27% (169 err.)
EfficientNetB0	96.63% (153 err.)	96.56% (156 err.)	96.67% (151 err.)	96.52% (158 err.)
ShuffleNetV2 (ImageNet)	94.33% (257 err.)	94.29% (259 err.)	94.18% (264 err.)	94.27% (260 err.)

Predicciones distintas imagen por imagen (de 4,534):

Modelo	Pi 4 vs Pi 5	Pi 4 vs laptop (`.tflite`)	Pi 4: 1 hilo vs 4 hilos
MobileNetV3-Large	27	**1**	0
EfficientNetB0	15	17	0
ShuffleNetV2 (ImageNet)	11	**1**	0

- **El reporte de la Pi 5 atribuía estas diferencias a "x86 frente a ARM".** Con la Pi 4 se ve que no es así: la Pi 4 (ARM) coincide casi exactamente con la laptop (x86) en MobileNetV3 y ShuffleNetV2 (1 diferencia cada uno), y es la **Pi 5** la que se separa. La explicación más probable es el conjunto de instrucciones del CPU. El runtime (XNNPACK) elige kernels distintos para las capas con cuantización dinámica según lo que soporte el procesador. La Pi 5 (ARMv8.2, con `SDOT` y FP16) usa kernels que redondean distinto, y eso cambia la

predicción en imágenes que están en la frontera entre dos clases. EfficientNetB0 difiere de ambos entornos, posiblemente porque usa capas que la laptop y la Pi 4 tampoco ejecutan igual.

- En la práctica, el efecto es pequeño: las cuatro versiones quedan a menos de 0.35 puntos entre sí, dentro del intervalo de confianza, y las conclusiones no cambian. Pero confirma que **las cifras oficiales deben medirse en el dispositivo final**, porque el mismo `.tflite` no da resultados idénticos en todos los procesadores.
- El número de hilos no afecta las predicciones: 0 diferencias entre 1 y 4 hilos, igual que en la Pi 5.

4.3 Recursos y estabilidad

	Pi 5	Pi 4
RAM del modelo (MobileNetV3 / EfficientNetB0 / ShuffleNetV2)	17.1 / 34.2 / 12.2 MB	16.9 / 33.9 / 12.0 MB
CPU en inferencia continua (1 hilo)	25%	25%
Deriva de latencia en 5 min (máx. absoluta)	0.01%	0.82%
Temperatura máxima	57.9 °C	50.6 °C
Throttling / bajo voltaje	ninguno	ninguno
Preprocesamiento por imagen	8.2 ms	19.3 ms

La memoria es la misma en las dos placas. La Pi 4 se calienta menos porque su CPU trabaja a menor frecuencia, aunque no tiene ventilador registrado en el sistema.

5. Cumplimiento de H1 en la Pi 4

Modelo	Latencia media	Peor caso	Margen (peor caso)	Peor caso + preprocesamiento	¿Cumple H1?
MobileNetV3-Large	52.28 ms	84.75 ms	×2.4	≈ 104 ms	Sí, con margen
EfficientNetB0	131.12 ms	169.43 ms	×1.2	≈ 189 ms	Sí, pero al límite
ShuffleNetV2 (ImageNet)	38.15 ms	53.76 ms	×3.7	≈ 73 ms	Sí, con margen

En la Pi 5 todos los modelos cumplían H1 con un margen de al menos 4.4 veces. En la Pi 4 la hipótesis se sigue cumpliendo, pero EfficientNetB0 queda tan cerca del límite que cualquier carga adicional del sistema (cámara, interfaz, actuadores) podría hacerlo fallar.

6. Prueba complementaria: 4 hilos de inferencia

Se repitió la prueba con `--num-threads 4`: mismas 4,534 imágenes y 2 minutos de estabilidad por modelo, igual que en la Pi 5. La corrida fue el 14/10/2026 de 00:40 a 01:03. Datos crudos: `reports/benchmark_rpi4/resultados_rpi4_4hilos.json`.

Modelo	Latencia 1 hilo	Latencia 4 hilos	Aceleración (Pi 5)	Cuadros/s 4 hilos	Desv. est. (1 → 4 hilos)	Máx 4 hilos	CPU 4 hilos	Temp. máx 4 hilos
MobileNetV3-Large	52.28 ms	32.30 ms	×1.62 (×1.55)	31.0	1.54 → 2.81 ms	74.21 ms	94.7%	48.2 °C
EfficientNetB0	131.12 ms	90.68 ms	×1.45 (×1.57)	11.0	2.61 → 3.25 ms	142.71 ms	92.6%	47.7 °C
ShuffleNetV2 (ImageNet)	38.15 ms	23.50 ms	×1.62 (×1.34)	42.5	0.86 → 2.13 ms	60.63 ms	96.2%	48.7 °C

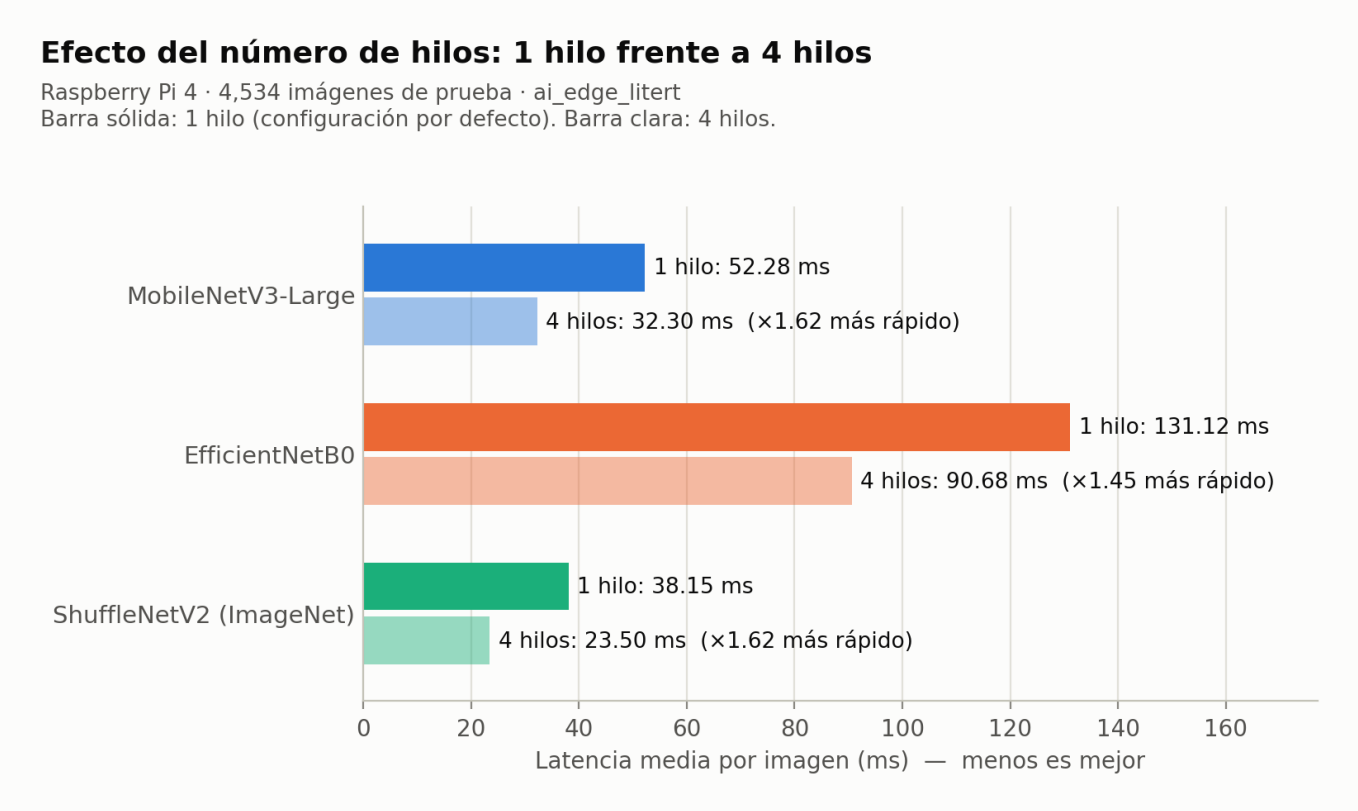


Figura 14. 1 hilo vs 4 hilos

- Con 4 núcleos la latencia mejora entre 1.45 y 1.62 veces, un poco más que en la Pi 5 en MobileNetV3 y ShuffleNetV2.
- **En la Pi 4 los 4 hilos sí pueden ser útiles:** es la única forma de que MobileNetV3 pase de los 30 cuadros/s (31.0). El precio es el mismo que en la Pi 5: la CPU sube de 25% a ~95% y no queda núcleo libre para la cámara, la interfaz y los actuadores.
- Las predicciones son idénticas con 1 y con 4 hilos y no hubo throttling (`0x0`).

- **Recomendación para la Pi 4:** usar 1 hilo si 19 cuadros/s bastan, lo cual es lo esperable en una banda o estación de clasificación donde los objetos pasan de uno en uno. Si se necesita más, probar con 2 hilos antes que con 4 (no se midió), para dejar núcleos libres.

7. Conclusiones

1. **Los tres modelos funcionan en la Raspberry Pi 4:** cumplen H1, son estables en ejecución continua, no tienen fugas de memoria y no provocan throttling.
2. **MobileNetV3-Large sigue siendo el modelo recomendado.** Tiene la misma precisión que EfficientNetB0 (diferencia no significativa, p = 0.42), es 2.5 veces más rápido (52.3 frente a 131.1 ms), usa la mitad de RAM y cumple H1 con margen aun sumando el preprocesamiento (≈ 104 ms en el peor caso).
3. **EfficientNetB0 no es viable en la práctica en una Pi 4.** Cumple H1 solo por 11 ms en el peor caso con preprocesamiento y apenas da 7.6 cuadros/s.
4. **ShuffleNetV2 pierde parte de su ventaja en la Pi 4.** Ahora es solo 1.37 veces más rápido que MobileNetV3 (en la Pi 5 era 1.66) y sigue perdiendo 2 puntos de precisión con una diferencia significativa.
5. **La Pi 5 es unas 3.4 veces más rápida que la Pi 4 con el modelo recomendado** (15.5 frente a 52.3 ms). La Pi 4 sirve como hardware mínimo, con menos holgura para la cámara y el resto del sistema. Si se usa una Pi 4, conviene medir el sistema completo (cámara + inferencia + actuadores) antes de darlo por bueno.
6. **El mismo `.tflite` da predicciones ligeramente distintas según el procesador.** La Pi 4 coincide con la laptop x86 y la Pi 5 se separa de ambas. Las diferencias son de décimas de punto y no cambian ninguna conclusión, pero confirman que las cifras oficiales deben medirse en el dispositivo final.

8. Evidencias generadas

Todas provienen de la ejecución en la Raspberry Pi 4.

Gráficas (`reports/figures/raspberry4/`): las mismas 14 del reporte de la Pi 5, generadas con `tests/graficar_benchmark_rpi.py` a partir de los JSON de la Pi 4.

Archivo	Contenido
`01_accuracy_modelos.png`	Accuracy de cada modelo con su intervalo de confianza del 95%
`02_accuracy_por_clase.png`	Accuracy por clase y modelo
`03_matrices_confusion.png`	Matrices de confusión de los tres modelos
`04_latencia_media.png`	Latencia media y cuadros por segundo
`05_latencia_vs_limite_H1.png`	Peor latencia de cada modelo frente al límite de 200 ms
`06_distribucion_latencia.png`	Histograma de la latencia por imagen con percentiles
`07_latencia_por_imagen.png`	Latencia de cada una de las 4,534 inferencias
`08_estabilidad_series.png`	Latencia, CPU, temperatura y RAM durante 5 minutos continuos
`09_precision_vs_latencia.png`	Compromiso precisión–velocidad–tamaño
`10_recursos.png`	Tamaño, RAM, CPU, inferencias/s, temperatura y tiempo de carga
`11_latencia_1_vs_4_hilos.png`	Efecto de usar 4 hilos de inferencia
`12_ejemplos_predicciones.png`	Las mismas 10 imágenes de ejemplo del reporte de la Pi 5, con las predicciones de la Pi 4
`13_errores_comunes.png`	Imágenes que los tres modelos clasificaron mal
`14_coincidencia_errores.png`	Cuántos errores comparten los modelos

Capturas de pantalla del proceso (`reports/figures/raspberry4/capturas/`): las pantallas de `benchmark_visual.py` en la corrida oficial de la Pi 4. Los SVG originales están en `reports/benchmark_rpi4/capturas_svg/` .

Archivo	Etapa
`captura_0a_sistema_y_modelos.png`	Ficha del dispositivo y de los tres modelos
`captura_0b_imagenes_de_prueba.png`	Distribución de las imágenes de prueba
`captura_1a/2a/3a_latencia_*.png`	Prueba de latencia y precisión de cada modelo (al terminar)
`captura_1b/2b/3b_estabilidad_*.png`	Monitor de estabilidad de cada modelo (al terminar los 5 min)
`captura_1c/2c/3c_resumen_*.png`	Tabla de resumen y matriz de confusión de cada modelo

Archivo	Etapa
`captura_9a_comparacion_final.png`	Tabla comparativa final
`captura_9b_accuracy_por_clase.png`	Accuracy por clase

> **Nota sobre `9a` y `9b`:** la versión de `benchmark_visual.py` que se usó en la corrida oficial tenía escrito fijo el texto "Raspberry Pi 5" en los títulos de estas dos pantallas (las cifras sí eran de la Pi 4). Se corrigió el script para que use la placa detectada y estas dos capturas se volvieron a generar con la misma función del script, a partir de `resultados_rpi4_1hilo.json`. Se verificó que el texto es idéntico al original salvo el nombre de la placa. Los SVG originales se conservan en `reports/benchmark_rpi4/capturas_svg/originales_titulo_pi5/`. Las otras 11 capturas no tenían ese texto y son las originales.

Capturas del escritorio de la Raspberry Pi 4 (`reports/figures/raspberry4/escritorio/`): capturas de pantalla reales (`grim`, 1920×1080) del escritorio de la Pi 4 mientras `benchmark_visual.py` corría en una terminal. Son de una ejecución de demostración posterior a las corridas oficiales, ya con el script corregido, con 1,500 imágenes, 30 s de estabilidad y 5 s de enfriamiento, igual que la demostración de la Pi 5. Mientras se grababa, la terminal redibujaba la pantalla y se tomaba una captura cada 4 s, así que la latencia que aparece es un poco mayor (~54.4 ms en MobileNetV3 frente a 52.3 ms; el preprocesamiento también subió a ~39 ms por imagen) que la de la corrida oficial. **Las cifras oficiales son las de la sección 3.**

Archivo	Momento
`escritorio_1_inicio_sistema_y_modelos.png`	Inicio: ficha del dispositivo, modelos a evaluar y carga de imágenes
`escritorio_2_latencia_mobilenet.png`	MobileNetV3-Large clasificando imágenes en vivo
`escritorio_3_estabilidad_mobilenet.png`	Monitor de estabilidad (CPU por núcleo, RAM, temperatura, frecuencia)
`escritorio_4_latencia_shufflenet.png`	ShuffleNetV2 clasificando imágenes en vivo
`escritorio_5_comparacion_final.png`	Tablas comparativas al terminar

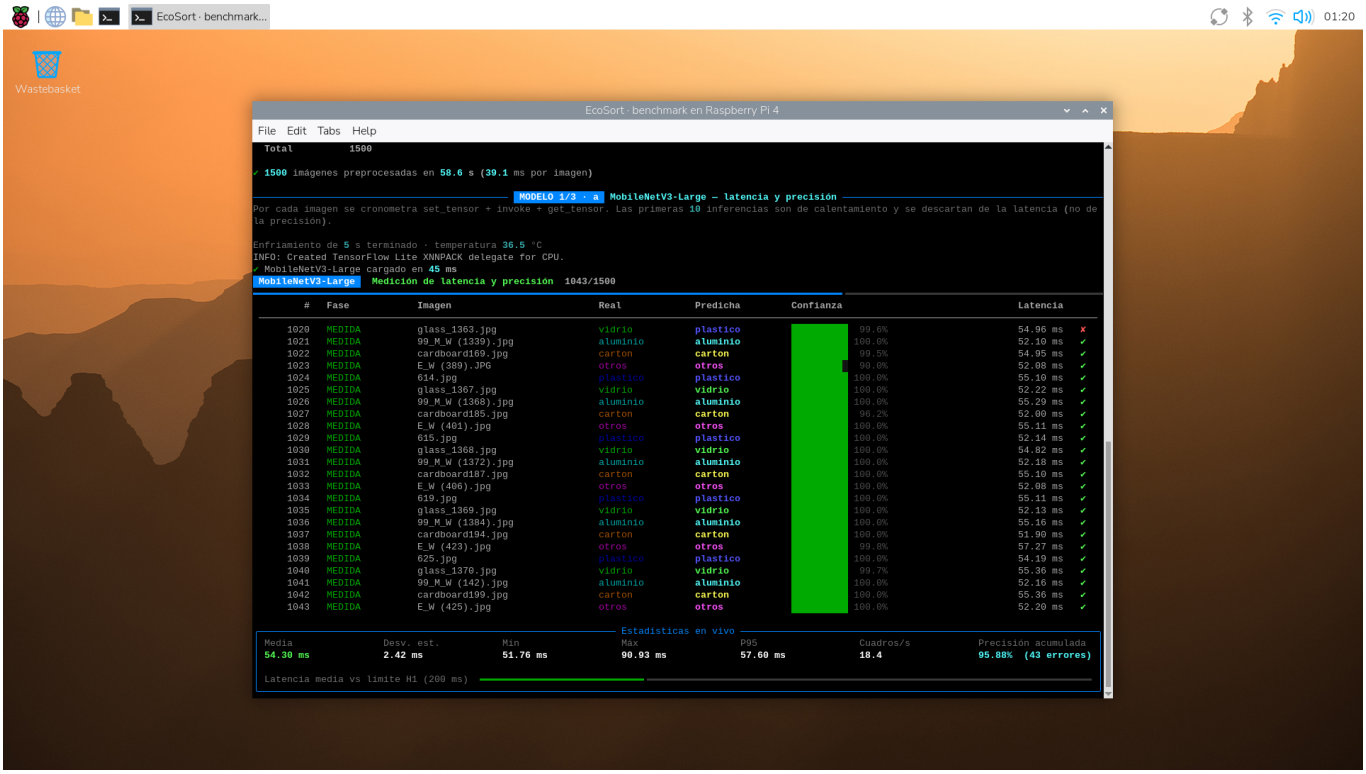


Figura 15. Raspberry Pi 4 ejecutando el benchmark

9. Cómo se hicieron y cómo reproducir las pruebas

Scripts utilizados:

- `tests/benchmark_visual.py`: el benchmark, el mismo de la Pi 5. Después de la corrida oficial solo se cambió para que los títulos muestren la placa detectada en lugar de "Raspberry Pi 5"; la medición no cambió.
- `tests/graficar_benchmark_rpi.py`: genera las gráficas y convierte las capturas. Se adaptó para tomar el nombre de la placa del JSON y para que las escalas funcionen con latencias más altas. Con los datos de la Pi 5 produce las mismas gráficas que antes.
- `tests/generar_pdf_reporte.py`: convierte este reporte a PDF. Se adaptó para recibir el `.md`, el PDF de salida y el JSON por parámetro.

En la Raspberry Pi 4 (carpeta `~/ecosort_benchmark/` con `benchmark_visual.py`, `modelos/`, `test_set/` y `venv/`):

```
python3 -m venv --system-site-packages venv
venv/bin/pip install --no-cache-dir ai-edge-litert==2.2.0
venv/bin/python benchmark_visual.py --salida-dir resultados_1hilo                          # corrida
oficial (~37 min)
venv/bin/python benchmark_visual.py --num-threads 4 --duracion-estabilidad 120 \
        --salida-dir resultados_4hilos                                                    # prueba con
4 hilos (~22 min)
venv/bin/python benchmark_visual.py --max-imagenes 1500 --duracion-estabilidad 30 \
        --enfriamiento 5 --salida-dir demo_escritorio                                     #
demostración para capturas
```

En la laptop:

```
# Gráficas y capturas
venv/bin/python tests/graficar_benchmark_rpi.py \
    --resultados reports/benchmark_rpi4/resultados_rpi4_1hilo.json \
    --resultados-hilos reports/benchmark_rpi4/resultados_rpi4_4hilos.json \
    --capturas reports/benchmark_rpi4/capturas_svg \
    --seleccion reports/benchmark_rpi4/seleccion_mosaicos.json \
    --salida reports/figures/raspberry4

# PDF
uv run --no-project --python 3.12 --with weasyprint --with markdown tests/generar_pdf_reporte.py \
    --md docs/reporte_benchmark_rpi4.md --salida docs/reporte_benchmark_rpi4.pdf \
    --datos reports/benchmark_rpi4/resultados_rpi4_1hilo.json \
    --subtitulo "Comparación de MobileNetV3-Large, EfficientNetB0 y ShuffleNetV2 (ImageNet) en la
Raspberry Pi 4 Model B, con la Raspberry Pi 5 como referencia"
```